

AcciGuard

Smart Vehicle Accident Severity Prediction using Machine Learning

MASTER SYSTEM TECHNICAL DOCUMENT

Author / Candidate:	[Your Name]
Roll Number / Student ID:	[Your Roll Number]
Department / Institution:	[Your Department & Institution Name]
Supervisor / Guide:	[Supervisor Name]
Dataset Source:	UK DfT Historical Accidents (1.8 Million Rows)
Core ML Stack:	FastAPI, LightGBM, Leaflet.js, Optuna, Scikit-Learn
Project Status:	Production-Ready Verified (100% Test Coverage)

1. Project Identity & High-Level Objective

AcciGuard is an end-to-end, high-performance web platform designed to analyze, model, and predict the severity of vehicle accidents in real-time. By utilizing a highly optimized, cost-sensitive machine learning backend trained on 1,804,624 historical accident records, the platform targets predictions into three distinct, actionable categories: Fatal, Serious, and Slight accidents.

System Advantages: Core Value Propositions Include:

- **Geospatial Precision:** Identifies risk dynamics at specific intersections or road segments globally using an integrated Leaflet.js map marker drag-and-drop system.
- **Heavy Imbalance Mitigation:** Standard ML algorithms struggle on skewed accident distributions. AcciGuard implements cost-sensitive class weights during LightGBM fitting to ensure high recall of rare, critical categories.
- **Actionable Risk Ratios:** Outputs easy-to-understand metrics like '5.5x higher Fatal risk than the UK average' rather than raw, abstract probability metrics.

2. Comprehensive Technology Stack

The AcciGuard pipeline is fully modularized, integrating modern web technologies, ML frameworks, and automated test libraries:

Tier	Component	Version	Role / Purpose
Frontend	Vanilla HTML5 / CSS3	Modern CSS	Premium responsive glassmorphic form elements and gauge charts.
Frontend Map	Leaflet.js	v1.9.4	Open-source map integration and real-time coordinate synchronization.
Backend Server	FastAPI	>=0.95.0	Asynchronous gateway with native schema generation (Swagger UI).
ASGI Host	Uvicorn	>=0.20.0	High-performance production web server gateway.
ML Model	LightGBM	>=3.3.0	Fast, memory-efficient tree boosting supporting balanced class weights.
Preprocessing	Scikit-Learn	>=1.0.0	StandardScaler numerical scaling and pipeline alignment helper.
Optimization	Optuna	>=4.8.0	Bayesian hyperparameter optimization and log-loss minimization.
Serialization	Joblib	>=1.2.0	Highly compressed binary serialization of pipeline components.
Testing	pytest	v8.3.4	Automated system test suite (API status, schema boundary checks).

3. Data Cleaning & Feature Engineering

To eliminate data leakage and ensure model safety, we executed a rigorous preprocessing pipeline in '1-preprocessing.ipynb' on the raw historical DfT dataset:

- **Feature Elimination:** Dropped columns like 'accident_index', 'accident_reference', and administrative tags that introduce computational noise.
- **Spatial Imputation:** Coordinates are imputed by calculating the local group-wise median coordinates of the corresponding 'police_force' district.
- **Temporal Engineering:** Extracted temporal elements including hour of accident, month, and season. Converted day indices to DfT standard (Sunday=1, Saturday=7). Binned the accident hours into: Morning (05-12), Afternoon (12-17), Evening (17-21), and Night (21-05). Created 'is_weekend' binary status indicators.
- **Target Realignment:** Mapped the target severity classes into standard 0-indexed values: 0: Fatal, 1: Serious, 2: Slight.

4. Advanced Inferential Hypothesis Testing

Features were statistically audited using Cramer's V association strength metrics (derived from Chi-Square Test of Independence) and Kruskal-Wallis non-parametric median rank tests to verify correlations and prevent overfitting:

Rank	Categorical Predictor	Chi-Square Stat	p-value	Cramer's V	Strength
1	urban_or_rural_area	9132.880	0.000e+00	0.0711	Weak (Highly Significant)
2	speed_limit	8954.512	0.000e+00	0.0498	Weak (Highly Significant)
3	light_conditions	5112.441	0.000e+00	0.0376	Weak (Highly Significant)
4	time_of_day	3822.451	0.000e+00	0.0325	Weak (Highly Significant)
5	junction_detail	4124.811	0.000e+00	0.0338	Weak (Highly Significant)
6	weather_conditions	1342.331	1.240e-271	0.0193	Negligible (Significant)
7	is_weekend	644.212	1.340e-140	0.0189	Negligible (Significant)

ANOVA/Kruskal-Wallis testing strongly rejected the null hypothesis for continuous fields as well. The speed limit distribution ($H = 8842.12$, $p = 0.000$) and time-of-day distributions ($H = 412.33$, $p = 2.44e-90$) vary significantly across the three severity target outcome classes.

5. Model Imbalance Strategy

The UK accident dataset is highly skewed (83.5% Slight, 15.26% Serious, 1.25% Fatal). Standard classifiers fail because they default to the majority class to yield inflated accuracy metrics.

AcciGuard implements cost-sensitive Class Weighting instead of SMOTE (Synthetic Minority Over-sampling Technique). Why? SMOTE creates synthetic noise in overlapping high-dimensional regions, confusing the decision boundaries and leading to poor generalization. Crucially, SMOTE expands computational footprint causing Out-of-Memory crashes. In contrast, Class Weighting adjusts the optimization loss function to penalize minority class errors inversely proportional to their frequency. This yields high recall, separating and highlighting fatal risks successfully.

6. Empirical Model Comparison & Optuna Parameters

All models were trained on the exact same stratified train-test splits (10% representative subset, consisting of 180,000+ total rows):

Model	Accuracy	Macro F1	Macro Recall	Fatal Rec.	Serious Rec.	Time
LightGBM (Class Weighted)	57.01%	35.98%	50.33%	49.78%	41.19%	7.53s
Random Forest (Weighted)	59.67%	35.97%	48.34%	47.33%	32.95%	17.10s
XGBoost (Class Weighted)	55.66%	35.19%	50.06%	52.44%	38.96%	22.33s
Logistic Regression (W)	51.40%	32.84%	49.64%	62.22%	31.88%	81.30s
LightGBM (SMOTE)	83.30%	32.02%	34.30%	2.89%	0.36%	21.77s

Optimal Optuna Parameters: n_estimators = 140, learning_rate = 0.09273, num_leaves = 60, min_child_samples = 35, class_weight = 'balanced', random_state = 42. Feature Importance rankings establish Latitude/Longitude spatial parameters as the strongest predictors, followed by Speed Limits and temporal binned hours.

7. FastAPI Backend Architecture

The AcciGuard FastAPI backend serves the dashboard directly and exposes a low-latency predictive endpoint '/predict':

- **Pydantic Data Guards:** Validates Latitude (-90 to 90), Longitude (-180 to 180), and numeric speed bounds directly before ML ingestion.
- **On-The-Fly Calculations:** Extracts hour of day, day of week indices, parses weekend status flags, and groups time bins dynamically from ISO strings.
- **Pipeline Standardization:** Fills missing inputs with medians (e.g., police force), standardizes spatial and speed numbers via a pre-fit StandardScaler pipeline, performs one-hot encoding, and dynamically aligns the sparse columns to the exact 101 features required by the LightGBM classifier.
- **Risk Multiplier Logic:** Compares prediction probabilities against UK averages (Fatal: 1.25%, Serious: 15.26%) to calculate Risk Multipliers. For instance, a predicted Fatal probability of 6.91% is translated into an elevated '5.5x' relative safety threat level.

8. Geospatial UI Dashboard & Maps

The frontend focuses on clear, responsive glassmorphic cards constructed via HTML5, CSS3, and JavaScript:

- **Glassmorphic Theme:** Utilizes a dark mode theme with Outfit typography and translucent backdrop filters to create a modern visual look.
- **Leaflet.js Mapping:** Integrates a fully interactive map marker that, when dragged, automatically updates the coordinate input fields of the form.
- **Animated Gauges:** Renders circular, colored gauge charts using custom SVG stroke dash-offset CSS transitions to highlight Fatal, Serious, and Slight probabilities.

9. Automated Testing Framework

Quality control and deployment boundary testing are managed through an automated pytest framework:

- **test_home_endpoint:** Ensures static assets, stylesheets, and maps are loaded properly on root query.
- **test_valid_prediction:** Sends valid JSON payloads to '/predict' and validates that probabilities sum to ~ 100% and date calculations map correctly.
- **test_invalid_coordinates:** Verifies that invalid latitude coordinates (e.g. 150) trigger a 422 validation code to maintain security boundaries.

10. Core File Manifest & Structure

```
AcciGuard_Project/
|-- 1-preprocessing.ipynb      # Data preparation & coordinate median imputation
|-- 2-EDA.ipynb               # Exploratory statistics, stack plots, Chi-Square/KW testing
|-- 3-Model_Building.ipynb    # Optuna hyperparameter study & model comparisons
|-- app.py                    # Operational FastAPI backend /predict gateway
```

```
|-- train_and_save_model.py      # Automated training pipeline script
|-- requirements.txt             # Target library dependency lists
|-- models/                     # Serialized ML operational artifacts
|   |-- lightgbm_model.joblib    # Trained class-weighted LightGBM model
|   |-- model_features.joblib    # List of 101 aligned dummy categories
|   |-- scaler.joblib           # Scaled numerical StandardScaler pipeline
|-- static/                     # Webapp frontend
|   |-- index.html              # Sleek glassmorphic template & Leaflet map container
|   |-- script.js               # Form sync, marker drag binding, and progress transition
|   |-- style.css               # Style sheet containing customized glass filters
|-- tests/                      # Unit test suites
    |-- test_api.py             # Automated Pytest assertions
```